

# ใบงานที่ 9 การปรับแต่งโมเดลภาษา (LLM Fine-Tuning)

## วัตถุประสงค์การเรียนรู้

- เขียนคำสั่งภาษา Python เพื่อเตรียมข้อมูลและตั้งค่าการปรับแต่งโมเดล (Fine-Tune) สำหรับสร้างแชทบอทเฉพาะทาง จากชุดข้อมูลที่กำหนดให้ ได้อย่างถูกต้องตามขั้นตอน
- เขียนคำสั่งภาษา Python เพื่อทดสอบและประเมินผลความแม่นยำของโมเดลที่ผ่านการปรับแต่ง จากชุดคำถามทดสอบที่สร้างขึ้น ได้อย่างถูกต้อง
- เขียนคำสั่งภาษา Python เพื่อผสานค่าน้ำหนัก (Model Merging) และบันทึกโมเดล (Export) สำหรับเตรียมนำไปใช้งานจริง ได้อย่างสมบูรณ์

## เครื่องมือและอุปกรณ์

- เครื่องคอมพิวเตอร์ที่มีการ์ดจอ (GPU) และติดตั้งโปรแกรม PyCharm IDE
- สิทธิ์การเข้าถึงโมเดลจาก Hugging Face (เช่น google/gemma-3-1b-it)
- ไฟล์ชุดข้อมูล dataset.json (ชุดข้อมูลถาม-ตอบเรื่องมะเขือเทศ)
- คอมพิวเตอร์ที่มี NVIDIA GPU ที่รองรับ CUDA
- เอกสารใบความรู้หน่วยที่ 9
- แบบฟอร์มบันทึกผลการปฏิบัติงาน (ท้ายใบงาน)

## ลำดับขั้นตอนการปฏิบัติงาน

คำชี้แจง: ให้ผู้เรียนเขียนโค้ดลงในโปรแกรม PyCharm ตามขั้นตอนต่อไปนี้ รันโปรแกรม และนำผลลัพธ์ที่ปรากฏบนหน้าจอ Terminal มาบันทึกลงในแบบฟอร์มท้ายใบงาน

### ขั้นตอนที่ 1: การปรับแต่งโมเดล (Fine-Tuning เพื่อสร้างแชทบอท)

- นำไฟล์ dataset.json มาวางในโฟลเดอร์โปรเจกต์
- สร้างไฟล์ Fine-tune.py เขียนโค้ดเพื่อโหลด Base Model แบบบีบอัด 4-bit (Quantization) เพื่อประหยัดทรัพยากรเครื่อง

3. เขียนคำสั่งตั้งค่า LoraConfig เพื่อสร้างแวนตาความรู้ (Adapter) โดยระบุ target\_modules ที่เกี่ยวข้องกับระบบความสนใจ (Attention) ของ AI
4. สั้ร้ันการฝึกสอน (Train) ด้วย Trainer และรอจนกระทั่งโปรแกรมบันทึกไฟล์เตอร์ gemma3-finetuned สำเร็จ

## ขั้นตอนที่ 2: การประเมินผลโมเดล (Model Evaluation)

1. สร้างไฟล์ evaluate\_model.py เพื่อโหลด Base Model มาสมทบกับ Adapter ที่เพิ่งฝึกสอนเสร็จ
2. สร้างชุดคำถามทดสอบ (Test Set) ที่มีทั้งคำถามในเนื้อหา (เช่น ลักษณะของมะเขือเทศ) และคำถามหลอกนอกเนื้อหา (เช่น การปลูกแตงโม)
3. รันโปรแกรมเพื่อทดสอบถาม-ตอบ และสังเกตพฤติกรรมการตอบคำถามของแชทบอท

## ขั้นตอนที่ 3: การบันทึกโมเดลเพื่อนำไปใช้งานจริง (Model Export)

1. สร้างไฟล์ export\_model.py เขียนคำสั่งโหลด Base Model แบบความละเอียดเต็ม 16-bit โดยบังคับให้โหลดเข้า CPU (device\_map="cpu") เพื่อป้องกัน VRAM เต็ม
2. โหลด Adapter มาสมทบ และใช้คำสั่ง merge\_and\_unload() เพื่อหลอมรวมค่าน้ำหนักความรู้ใหม่เข้ากับสมองหลักแบบถาวร
3. สั้บบันทึกโมเดล (Save) ไปยังไฟล์เตอร์ใหม่ เช่น gemma3-bot-final

## แบบบันทึกผลการปฏิบัติงาน

### ส่วนที่ 1: การตั้งค่าพารามิเตอร์สำหรับการปรับแต่งโมเดล (Fine-Tuning)

ตรงกับวัตถุประสงค์ข้อที่: 1 (เขียนคำสั่งภาษา Python เพื่อเตรียมข้อมูลและตั้งค่าการปรับแต่งโมเดล (Fine-Tune) สำหรับสร้างแชทบอทเฉพาะทาง จากชุดข้อมูลที่กำหนดให้ ได้อย่างถูกต้องตามขั้นตอน)

1. ในขั้นตอนการแปลงข้อมูล (Tokenize) ผู้เรียนได้ใส่สัญลักษณ์ใดต่อท้ายประโยคคำตอบ เพื่อสอนให้ AI รู้จักหยุดประมวลผลเมื่อตอบเสร็จ?  
ตอบ สัญลักษณ์ `<eos>` (End of Sequence) หรือ `<end_of_turn>` (ขึ้นอยู่กับค่า Tokenizer ของโมเดล Gemma) เพื่อเป็นการบอกให้โมเดลรู้ว่าสิ้นสุดการตอบคำถามแล้ว จะได้ไม่พ่นข้อความหรือแต่งประโยคอื่นต่อ
2. ในการตั้งค่า LoraConfig ผู้เรียนได้ระบุโมดูลเป้าหมาย (Target Modules) ในสมองของ AI เป็นส่วนใดบ้าง? (ระบุอย่างน้อย 3 โมดูล)  
ตอบ โมดูลในส่วนของ Attention Mechanism เช่น `q_proj` (Query), `k_proj` (Key), `v_proj` (Value) หรืออาจมี `o_proj`, `gate_proj`, `up_proj`, `down_proj` (สามารถตอบ 3 โมดูลใดก็ได้ในกลุ่มนี้)

### ส่วนที่ 2: ผลการประเมินแชทบอท (Evaluation)

ตรงกับวัตถุประสงค์ข้อที่: 2 (เขียนคำสั่งภาษา Python เพื่อทดสอบและประเมินผลความแม่นยำของโมเดลที่ผ่านการปรับแต่ง จากชุดคำถามทดสอบที่สร้างขึ้น ได้อย่างถูกต้อง)

1. จากการนำชุดคำถามทดสอบไปประเมินแชทบอท จงบันทึกพฤติกรรมการตอบสนองของโมเดลที่ปรากฏบน Terminal:
  - กรณีที่ 1 ถามคำถามที่ตรงกับข้อมูลใน dataset.json (เช่น สาเหตุที่ไบเมะเชื้อเทศม้วน) ผลลัพธ์การตอบของ AI คือ: AI สามารถตอบคำถามเกี่ยวกับมะเชื้อเทศได้ถูกต้อง ตรงประเด็น (เช่น อธิบายสาเหตุไบเมะม้วนได้เป๊ะตาม Dataset) และหยุดประโยคได้สมบูรณ์ ไม่พ่นข้อความขยะต่อ
  - กรณีที่ 2 ถามคำถามหลอกที่ไม่อยู่ในข้อมูล (เช่น "แต่งโมปลูกยังงัยคะ?") ผลลัพธ์การตอบของ AI คือ: AI ยังสามารถตอบวิธีการปลูกแต่งโม หรือให้ข้อมูลพื้นฐานเกี่ยวกับแต่งโมได้ แม้จะไม่ได้อยู่ใน Dataset เรื่องมะเชื้อเทศก็ตาม

- จากผลลัพธ์ในกรณีที่ 2 การที่ AI สามารถตอบเรื่องอื่นนอกเหนือจากมะเขือเทศได้ สะท้อนให้เห็นถึงคุณลักษณะใดของ Base Model ก่อนนำมาทำ Fine-Tuning?

ตอบ สะท้อนให้เห็นถึง "ความรู้พื้นฐาน" (General Knowledge หรือ Pre-trained Knowledge) ที่ตัว Base Model ได้เรียนรู้มาจากข้อมูลมหาศาลบนอินเทอร์เน็ตมาก่อนหน้า การทำ Fine-Tuning แบบ LoRA เป็นเพียงการ "เพิ่มความเชี่ยวชาญเฉพาะด้าน" เข้าไป โดยไม่ได้ไปลบหรือทำลายความรู้รอบตัวเดิมที่โมเดลมีอยู่

### ส่วนที่ 3: การผสานค่าน้ำหนักและการบันทึกโมเดล (Model Merging & Export)

ตรงกับวัตถุประสงค์ข้อที่: 3 (เขียนคำสั่งภาษา Python เพื่อผสานค่าน้ำหนัก (Model Merging) และบันทึกโมเดล (Export) สำหรับเตรียมนำไปใช้งานจริง ได้อย่างสมบูรณ์)

- ก่อนทำคำสั่งผสานโมเดล เหตุใดผู้เรียนจึงต้องกำหนดพารามิเตอร์ `device_map="cpu"` แทนการโหลดเข้าการ์ดจอ (GPU) เหมือนตอนเทรน?

ตอบ เพื่อ ป้องกันปัญหาหน่วยความจำการ์ดจอเต็ม (VRAM Out of Memory) เนื่องจากตอนเทรนเราโหลดโมเดลแบบบีบอัด (4-bit) แต่ขั้นตอนการ Merge Model เพื่อ Export นั้น จำเป็นต้องโหลด Base Model คืนร่างเป็นความละเอียดสูง (16-bit หรือ 32-bit) ซึ่งใช้พื้นที่มหาศาล การย้ายไปคำนวณบน CPU ที่ใช้ RAM ของเครื่อง (ซึ่งมักจะมีขนาดใหญ่กว่า VRAM ของการ์ดจอมาก) จึงปลอดภัยและช่วยหลีกเลี่ยงอาการโปรแกรมค้างหลุดได้

### ส่วนที่ 4: การทดลองปรับแต่งโค้ด (Coding Challenges)

ตรงกับวัตถุประสงค์ข้อที่: ครอบคลุมข้อ 1 และ 2 (การทดลองปรับแต่งไฮเปอร์พารามิเตอร์และวิเคราะห์ผลกระทบที่มีต่อความแม่นยำของโมเดล)

คำชี้แจง: ให้ผู้เรียนกลับไปแก้ไขโค้ดตามโจทย์ที่กำหนด สั่งรันโปรแกรมใหม่ และบันทึกผลลัพธ์ที่เปลี่ยนไป

- กลับไปไฟล์ `Fine-tune.py` ลองปรับค่า `num_train_epochs` จากเดิมให้เหลือเพียง 1 รอบ สังเกตเรนใหม่จนเสร็จ จากนั้นนำโมเดลไปทดสอบด้วยคำถามเดิมในไฟล์ `evaluate_model.py`
  - ผลลัพธ์: แชทบอทยังคงตอบคำถามเรื่องมะเขือเทศได้แม่นยำและหยุดประโยคได้สมบูรณ์เหมือนตอนเทรน 5 รอบหรือไม่? อย่างไร?

ตอบ แชทบอทจะ ตอบไม่แม่นยำและอาจหยุดประโยคไม่สมบูรณ์ (เกิดภาวะ Underfitting) เพราะ AI มีรอบการเรียนรู้ (Epoch) น้อยเกินไป ทำให้ยังจับรูปแบบความสัมพันธ์ของคำถาม-คำตอบ และสัญลักษณ์การหยุด (<eos>) จากชุดข้อมูลใหม่ไม่ได้

2. กลับไปที่ไฟล์ `evaluate_model.py` ในส่วนของการสร้างคำถาม ลองลบการจัดรูปแบบทิ้ง โดยเปลี่ยนจาก `prompt = f"คำถาม: {q}\nตอบ:"` ให้เหลือเพียงการส่งตัวแปรคำถามเพียง ๑ เข้าไป คือ `prompt = q` แล้วสั่งรันโปรแกรมใหม่
- ผลลัพธ์: พฤติกรรมการตอบของ AI เปลี่ยนไปอย่างไร? (ยังคงตอบสั้น ๆ แบบผู้เชี่ยวชาญ หรือกลับไปมีพฤติกรรมพ่นน้ำลายถามเองตอบเองยาวเหยียดเหมือน Base Model?)
- ตอบ พฤติกรรมการตอบของ AI จะเปลี่ยนไป โดย อาจจะกลับไปพ่นข้อความยาวเหยียดพยายามแต่งประโยคต่อ หรือถามเองตอบเอง เพราะการทำ Fine-tuning โมเดลจะจดจำ "รูปแบบบริบท" (Format) ไปด้วย หากเราป้อนข้อมูลดิบ ๆ ที่ไม่มีโครงสร้าง "คำถาม-คำตอบ" กำกับ AI จะสับสนและคิดว่านี่คือการเขียนบทความต่อให้จบ แทนที่จะเป็นการตอบคำถาม
3. ในไฟล์ `evaluate_model.py` ตรงบรรทัดที่สั่ง `model.generate(...)` ให้ลองปรับหรือเพิ่มพารามิเตอร์จำกัดความยาวเป็น `max_new_tokens=5` แล้วสั่งรันโปรแกรม
- ในไฟล์ `evaluate_model.py` ตรงบรรทัดที่สั่ง `model.generate(...)` ให้ลองปรับหรือเพิ่มพารามิเตอร์จำกัดความยาวเป็น `max_new_tokens=5` แล้วสั่งรันโปรแกรม
- ตอบ AI จะถูกบังคับให้สร้างคำศัพท์ออกมาได้สูงสุดแค่ 5 คำ (Tokens) ส่งผลให้ ประโยคคำตอบ ถูกตัดจบกลางคันอย่างกะทันหัน (เช่น ตอบมาแค่ "โบมะเชื้อเทศมันเกิดจาก...") ทำให้ผู้ใช้ได้รับคำตอบที่ไม่สมบูรณ์และอ่านไม่รู้เรื่อง